



Escuela de Administración de Tecnologías de Información

Bases de Datos – TI3600

Proyecto #1

Intervención sobre una Base de Datos Existente

Estudiante:

Gabriel Gutiérrez Brenes

Profesora:

Ing. Jacqueline Solís Céspedes

II Semestre

2026

Índice

Parte 1: Ampliación del Sistema.....3

Parte 3.A: Documentación y Trazabilidad.....11

Parte 3.B: Evidencias visuales de interfaz gráfica.....23

Parte 3.C: Repositorio en Github.....26

Bibliografía.....27

Anexos.....28

PARTE 1: Ampliación del sistema**Módulo de Gestión de Ubicaciones**

RF-08	Gestión y Registro de Ubicaciones
Descripción	<i>El sistema deberá permitir la administración de las ubicaciones físicas donde se llevan a cabo los eventos desde la interfaz gráfica. Cada ubicación deberá almacenar de forma obligatoria los siguientes atributos desde la interfaz:</i> • <i>id_ubicacion: Identificador único autoincrementable de la ubicación.</i> • <i>nombre: Nombre descriptivo del lugar (ej. Auditorio Principal, Sala de Conferencias B).</i> • <i>direccion: Dirección física o señas exactas del recinto.</i> • <i>ciudad: Ciudad o localidad donde se encuentra ubicada.</i> • <i>capacidad: Número máximo de personas que soporta el recinto.</i>
Modelo Conceptual	El modelo conceptual se compone de la entidad 'ubicación' (rectángulo) y de cinco atributos (óvalos); nombre, dirección, ciudad, capacidad y id_ubicacion, este último subrayado para identificar que es la llave primaria (PK). 
Modelo Lógico	ubicacion (id_ubicacion[PK], nombre, dirección, ciudad, capacidad)
Modelo Físico	Para el código SQL, se crea la tabla mediante el comando CREATE TABLE, luego se crea cada atributo especificando sus características: el id_ubicacion se denota como llave primaria de la entidad y se añade SERIAL para especificar el incremento en cada inserción. El nombre, ciudad y la dirección corresponden a una cadena de no más de la cantidad dada de caracteres y de tipo VARCHAR para no guardar específicamente los espacios del paréntesis y se utilicen los necesarios (el 50 es el límite). La capacidad es un número, por tanto, corresponde al tipo INT. Todos los valores se acompañan del término NOT NULL para no permitir la inserción de datos vacíos. <pre>CREATE TABLE ubicaciones (id_ubicacion SERIAL PRIMARY KEY, nombre VARCHAR(50) NOT NULL, direccion VARCHAR(100) NOT NULL, ciudad VARCHAR(30) NOT NULL, capacidad INT NOT NULL);</pre>
Código Python	La gestión de ubicaciones inicia en la línea 663, se utilizan las siguientes funciones: <pre>configurar_pestana_ubicaciones ubicación_seleccionado_id cargar_ubicacion_seleccionado limpiar_form_ubicacion agregar_ubicacion actualizar_ubicacion eliminar_ubicacion cargar_datos_ubicaciones</pre>

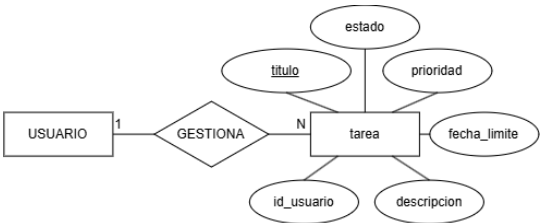
RF-09	Asociación Espacial de Eventos
Descripción	La entidad Eventos incorporará una clave foránea (<i>id_ubicacion</i>) que vincula de manera directa cada evento con un recinto específico de la base de datos, permitiendo estructurar consultas complejas desde la interfaz orientadas a: • Consultar el histórico y la agenda actual de eventos que se realizan en cada ubicación específica. • Identificar cuáles ubicaciones registran un mayor volumen de eventos programados. • Detectar y prevenir la programación de eventos simultáneos o concurrentes en una misma ubicación física para evitar conflictos de espacio.
Modelo Conceptual	La relación que comparten las entidades de usuario y evento es UBICACIÓN-PERTENECE-EVENTO (1:N), para permitir la asociación de un evento a la ubicación donde se realizará. En general, el fragmento de la entidad ubicación con sus atributos y relaciones se muestra así: <pre>graph LR evento[evento] --- N --- PERTENECE{PERTENECE} PERTENECE --- 1 --- ubicacion[ubicacion] ubicacion --- id_ubicacion((id_ubicacion)) ubicacion --- nombre((nombre)) ubicacion --- direccion((direccion)) ubicacion --- ciudad((ciudad)) ubicacion --- capacidad((capacidad))</pre>
Modelo Lógico	Para el modelo lógico, se transforma el <i>id_ubicacion</i> (realizado en RF-08) como una llave foránea [FK], para que esté vinculada a la entidad de evento. eventos(<i>id_evento</i>[PK],<i>id_usuario_propietario</i>[FK],<i>id_categoria</i>,<i>titulo</i>,<i>descripcion</i>,<i>fecha_inicio</i>,<i>fecha_fin</i>)
Modelo Físico	En el código SQL, dado para la creación de la tabla 'eventos', se añade el comando REFERENCES donde se especifica la inclusión del <i>id:ubicación</i> como llave foránea a ese dato. Esto permitirá que la tabla guarde el dato que guarda la tabla usuarios. (se identifica en el modelo SQL con subrayado amarillo). <pre>CREATE TABLE eventos (id_evento SERIAL PRIMARY KEY, id_usuario_propietario INT NOT NULL REFERENCES usuarios(id_usuario), id_categoria INT NOT NULL REFERENCES categorias(id_categoria), titulo VARCHAR(100) NOT NULL, descripcion TEXT, fecha_inicio TIMESTAMP NOT NULL, fecha_fin TIMESTAMP NOT NULL, CONSTRAINT check_fechas CHECK (fecha_fin > fecha_inicio), id_ubicacion REFERENCES ubicaciones(id_ubicacion));</pre>

RF-10	Asociación Espacial de Eventos
Descripción	<i>La entidad Eventos incorporará una clave foránea (id_ubicacion) que vincula de manera directa cada evento con un recinto específico de la base de datos, permitiendo estructurar consultas complejas desde la interfaz orientadas a:</i> <i>• Consultar el histórico y la agenda actual de eventos que se realizan en cada ubicación específica.</i> <i>• Identificar cuáles ubicaciones registran un mayor volumen de eventos programados.</i> <i>• Detectar y prevenir la programación de eventos simultáneos o concurrentes en una misma ubicación física para evitar conflictos de espacio.</i>
Modelo Físico	Para la implementación del modelo físico, se utilizan los comandos de SELECT y FROM para direccionar la toma de datos en la tabla de ubicaciones para encontrar los nombres y permitir que se muestren en la sección de eventos. Para cada evento se muestra la ubicación donde se realizará. Además, se utilizó el comando ORDER BY para ordenar los datos de acuerdo al nombre. <pre>SELECT nombre FROM ubicaciones ORDER BY nombre</pre>
Código Python	La gestión de eventos inicia en la línea 455, se añadieron las funciones de: cargar_ub_combo

RF-11	Registro y Control de Disponibilidades
Descripción	<i>El sistema deberá incorporar un módulo de gestión de disponibilidad para optimizar la coordinación de agendas personales. Actualmente, si bien los usuarios tienen la capacidad de crear eventos, el modelo carece de un mecanismo formal para conocer los periodos de tiempo en los que una persona se encuentra libre u ocupada. Para dar solución a esto, se implementará la entidad disponibilidades, vinculada directamente al perfil de cada usuario, en conjunto con un catálogo de tipos_disponibilidad (tales como disponible, ocupado o no disponible). Cada franja de disponibilidad registrará de forma obligatoria el usuario asociado, la fecha correspondiente, la hora de inicio, la hora de fin y el estado o tipo que define su condición temporal, permitiendo a los usuarios administrar sus periodos de tiempo libres u ocupados mediante operaciones CRUD (crear, editar, consultar y eliminar franjas) desde la interfaz.</i>
Modelo Conceptual	Se define una entidad llamada 'disponibilidad' que se vincula a la tabla de usuarios. Esta nueva entidad contendrá los atributos de fecha, hora_inicio, hora_fin, tipo_disponibilidad; que representa la llave primaria y el id_usuario, que corresponde a una clave foránea a la tabla de usuarios. La disponibilidad se relaciona con el usuario en una cardinalidad de (1:N) es decir, 1 disponibilidad para N usuarios. La representación de modelo conceptual se verá así:  <pre> graph TD disponibilidad[disponibilidad] --- tipo_disponibilidad([tipo_disponibilidad]) disponibilidad --- fecha([fecha]) disponibilidad --- hora_inicio([hora_inicio]) disponibilidad --- id_usuario([id_usuario]) disponibilidad --- hora_fin([hora_fin]) </pre>
Modelo Lógico	disponibilidad(tipo_disponibilidad[PK], fecha, hora_inicio, hora_fin, id_ubicacion[FK])
Modelo Físico	Se crea la tabla 'disponibilidad' mediante CREATE TABLE. Se añaden los atributos y sus respectivas notaciones para establecer el tipo de sus datos: para el atributo 'fecha' se añade el tipo TIMESTAMP y se hace referencia de la clave foránea del id_ubicacion mediante REFERENCES ubicaciones(id_ubicacion). <pre> CREATE TABLE disponibilidad (tipo_disponibilidad VARCHAR(15) PRIMARY KEY, fecha_inicio TIMESTAMP NOT NULL, fecha_fin TIMESTAMP NOT NULL, hora_inicio TIMESTAMP NOT NULL, hora_fin TIMESTAMP NOT NULL, id_ubicacion REFERENCES ubicaciones(id_ubicacion)); </pre>
Código Python	La gestión de disponibilidad inicia en la línea 776, se crearon las funciones de (se usó 'dispon' como diminutivo de disponibilidad): <pre> configurar_pestana_dispon dispon_seleccionado_id cargar_dispon_seleccionado limpiar_form_dispon agregar_dispon actualizar_dispon eliminar_dispon </pre>

RF-11	Registro y Control de Disponibilidades
	cargar_datos_dispon

RF-12	Análisis de Solapamientos e Intervalos de Tiempo
Descripción	<i>El motor de base de datos deberá soportar consultas analíticas complejas sobre los rangos horarios para resolver problemáticas de concurrencia y planificación. Específicamente, el sistema estará capacitado para procesar intervalos de tiempo mediante la detección de cruces o traslapes temporales y las salidas se integrarán en la GUI. Esto permitirá resolver requerimientos críticos de negocio —como determinar de manera automática qué usuarios se encuentran libres y disponibles para una reunión o convocatoria en un rango horario específico (por ejemplo, entre las 2:00 p. m. y las 4:00 p. m.)— cruzando los compromisos ya agendados en los eventos con las franjas de disponibilidad declaradas por cada participante.</i>
Modelo Físico	Para identificar el rango de tiempo de los eventos que se traslapen con otros, se agregará el comando para seleccionar dos eventos, un evento 1 y otro 2. El análisis de este comando se basa en calcular si la hora de fin del evento 1 es mayor que la hora final del evento 2; por ejemplo, si el evento 1 finaliza a las 16:00 y el evento 2 comienza a las 14:00, es evidente que hay un traslape de 2 horas al calcular que $fin1 > inicio2 \Rightarrow 16 > 14$ es cierto. Para el comando, se utiliza JOIN el para unir la tabla en dos de sus eventos y se buscan aquellos que cumplan la condición de traslape ($fin1 > inicio2$). <pre>SELECT e1.id_evento, e2.id_evento FROM eventos e1 JOIN eventos e2 ON WHERE e1.fecha_fin > e2.fecha_inicio</pre>
Código Python	Se agregaron las siguientes líneas para enviar una consulta sobre el traslape de horarios: <pre>traslape = self.ejecutar_consulta(""" SELECT COUNT(*) FROM eventos WHERE fecha_fin > %s AND %s > fecha_inicio; """, (fecha_fin, fecha_inicio), fetch=True) if(traslape[0][0]>0): <u>messagebox</u>.showinfo("Aviso","Hay traslape de horario con otro evento.")</pre>

RF-15	Seguimiento y Control de Tareas por Evento
Descripción	El sistema ampliará sus capacidades de agenda hacia una herramienta de seguimiento mediante la incorporación de la entidad tareas, vinculada directamente a los eventos de la plataforma. Cada tarea contemplará atributos descriptivos como título, descripción, prioridad, fecha límite y un usuario responsable asignado. Asimismo, el ciclo de vida de cada tarea se administrará a través de un conjunto de estados predefinidos (Pendiente, En progreso, Completada y Cancelada). Cada usuario podrá realizar un CRUD (crear, leer, actualizar estados y eliminar) completo desde la interfaz.
Modelo Conceptual	Se crea la entidad de nombre 'tarea' con seis atributos: título, como llave primaria, prioridad, fecha_limite, descripción, id_usuario, como llave foránea y estado, que puede ser pendiente, en progreso, completada o cancelada. Dentro de la interfaz del sistema, el usuario será capaz de crear, leer, actualizar o eliminar tareas, por lo tanto, se establece una relación de GESTIONA con cardinalidad 1:N, donde 1 usuario puede gestionar N tareas: 
Modelo Lógico	tareas (titulo[PK],estado,prioridad,fecha_limite,descripcion,id_usuario[FK])
Modelo Físico	Para la creación de la nueva tabla, se usa el comando CREATE TABLE con el nombre de 'tareas'. En las siguientes líneas se colocan los atributos correspondientes: titulo, como llave primaria y de tipo VARCHAR con límite de 25 caracteres; estado, que puede ser pendiente, en progreso, completada o cancelada; fecha_limite como TIMESTAMP; descripción como VARCHAR, y una llave foránea que apunta al id_usuario del usuario que gestiona la tarea. <pre>CREATE TABLE tareas (titulo VARCHAR(25) PRIMARY KEY, estado VARCHAR(15) NOT NULL, prioridad VARCHAR(20) NOT NULL, fecha_limite TIMESTAMP NOT NULL, descripcion VARCHAR(50), id_usuario REFERENCES usuarios(id_usuario));</pre>
Código Python	La gestión de tareas inicia en la línea 907 del código, donde se implementaron las funciones de: <pre>configurar_pestana_tarea tarea_seleccionado_id cargar_tarea_seleccionado limpiar_form_tarea agregar_tarea actualizar_tarea eliminar_tarea cargar_datos_tarea</pre>

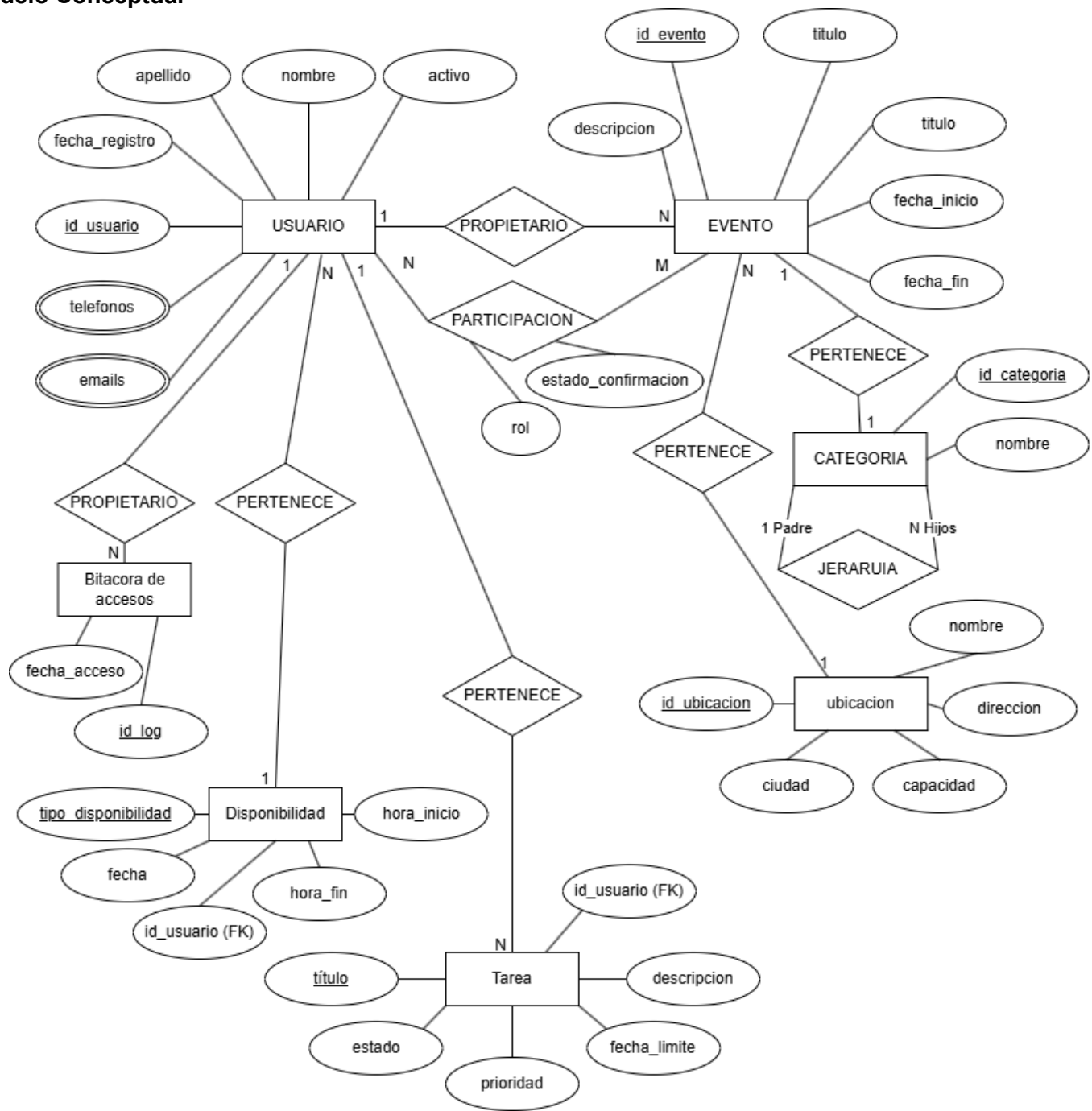
RF-16	Carga de Trabajo y Vencimientos
Descripción	<i>La base de datos deberá soportar consultas orientados a medir el rendimiento y control de pendientes, permitiendo resolver interrogantes críticas como la cantidad de tareas pendientes asignadas por usuario o la identificación de aquellos eventos que arrastran tareas vencidas fuera de su plazo límite y se debe desplegar en la pantalla de la aplicación.</i>
Modelo Físico	Primero, se utiliza el comando SELECT y FROM para seleccionar los datos de la tabla tareas. Luego se utiliza WHERE para detectar aquellas donde el atributo de estado sea igual a “pendiente”. Lo mismo se aplica para las tareas en que su estado sea “progreso” <pre>SELECT * FROM tareas WHERE estado = 'pendiente'; SELECT * FROM tareas WHERE estado = 'progreso';</pre>
Código Python	La configuración de la pestaña de estados de tareas inicia en la línea 1038, donde se implementaron las funciones de: <pre>configurar_pestana_estados_tareas cargar_datos_estados_tarea</pre>

RF-17	Reporte de Carga de Trabajo y Tareas Pendientes por Usuario
Descripción	<i>El sistema deberá integrar un reporte de seguimiento dentro de la interfaz del módulo de tareas, el cual detallará de forma cuantitativa el volumen de tareas activas (filtradas en estados como pendiente o en progreso) asignadas a cada miembro del equipo. Asimismo, la consulta destacará de manera específica aquellas tareas que se encuentren vencidas respecto a su fecha límite, transformando la agenda en una herramienta de control de productividad orientada a la detección temprana de cuellos de botella y sobrecargas de trabajo.</i>
Modelo Físico	Primero, se utiliza el comando SELECT y FROM para seleccionar los datos de la tabla tareas. Luego se utiliza WHERE para detectar aquellas donde el atributo de estado sea igual a "pendiente". Lo mismo se aplica para las tareas en que su estado sea "progreso" <pre>SELECT * FROM tareas WHERE fecha_limite < CURRENT_TIMESTAMP;</pre>
Código Python	La configuración de la pestaña de estados de tareas inicia en la línea 1038, donde se implementaron las funciones de: <pre>configurar_pestana_estados_tareas cargar_datos_estados_tarea</pre>

Parte 3.A: Documentación y Trazabilidad

1. Modelos de representación:

1.1 Modelo Conceptual



1.2 Modelo Lógico

Creación de tablas Ubicaciones, Disponibilidad y Tareas:

ubicacion (id_ubicacion[PK], nombre, dirección, ciudad, capacidad)

disponibilidad(tipo_disponibilidad[PK], fecha, hora_inicio, hora_fin, id_ubicacion[FK])

tareas (titulo[PK], estado, prioridad, fecha_limite, descripcion, id_usuario[FK])

Adición de id_ubicación a tabla Eventos:

eventos(id_evento[PK], id_usuario_propietario[FK], id_categoria, titulo, descripción, fecha_inicio, fecha_fin)

1.3 Modelo físico (SQL):

```
-- DROP SCHEMA prototipo;
```

```
CREATE SCHEMA prototipo AUTHORIZATION postgres;
```

```
-- DROP SEQUENCE prototipo.categorias_id_categoria_seq;
```

```
CREATE SEQUENCE prototipo.categorias_id_categoria_seq  
INCREMENT BY 1  
MINVALUE 1  
MAXVALUE 2147483647  
START 1  
CACHE 1  
NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE prototipo.categorias_id_categoria_seq OWNER TO postgres;  
GRANT ALL ON SEQUENCE prototipo.categorias_id_categoria_seq TO postgres;
```

```
-- DROP SEQUENCE prototipo.eventos_id_evento_seq;
```

```
CREATE SEQUENCE prototipo.eventos_id_evento_seq  
INCREMENT BY 1  
MINVALUE 1  
MAXVALUE 2147483647  
START 1  
CACHE 1  
NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE prototipo.eventos_id_evento_seq OWNER TO postgres;  
GRANT ALL ON SEQUENCE prototipo.eventos_id_evento_seq TO postgres;
```

```
-- DROP SEQUENCE prototipo.log_accesos_id_log_seq;
```

```
CREATE SEQUENCE prototipo.log_accesos_id_log_seq  
INCREMENT BY 1  
MINVALUE 1  
MAXVALUE 2147483647  
START 1  
CACHE 1  
NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE prototipo.log_accesos_id_log_seq OWNER TO postgres;  
GRANT ALL ON SEQUENCE prototipo.log_accesos_id_log_seq TO postgres;
```

```
-- DROP SEQUENCE prototipo.ubicaciones_id_ubicacion_seq;
```

```
CREATE SEQUENCE prototipo.ubicaciones_id_ubicacion_seq  
INCREMENT BY 1  
MINVALUE 1  
MAXVALUE 2147483647  
START 1  
CACHE 1  
NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE prototipo.ubicaciones_id_ubicacion_seq OWNER TO postgres;  
GRANT ALL ON SEQUENCE prototipo.ubicaciones_id_ubicacion_seq TO postgres;
```

```
-- DROP SEQUENCE prototipo.usuarios_id_usuario_seq;
```

```
CREATE SEQUENCE prototipo.usuarios_id_usuario_seq  
INCREMENT BY 1  
MINVALUE 1  
MAXVALUE 2147483647  
START 1  
CACHE 1  
NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE prototipo.usuarios_id_usuario_seq OWNER TO postgres;  
GRANT ALL ON SEQUENCE prototipo.usuarios_id_usuario_seq TO postgres;
```

```
-- prototipo.disponibilidad definition
```

```
-- Drop table
```

```
-- DROP TABLE prototipo.disponibilidad;
```

```
CREATE TABLE prototipo.disponibilidad (  
  tipo_disponibilidad varchar(15) NOT NULL,  
  fecha_inicio timestamp NOT NULL,  
  fecha_fin timestamp NULL,  
  hora_inicio timestamp NOT NULL,  
  hora_fin timestamp NOT NULL,  
  CONSTRAINT disponibilidad_fecha_not_null NOT NULL fecha_inicio,  
  CONSTRAINT disponibilidad_pkey PRIMARY KEY (tipo_disponibilidad),  
  CONSTRAINT disponibilidad_tipo_disponibilidad_not_null NOT NULL tipo_disponibilidad,  
  CONSTRAINT disponibilidad_hora_inicio_not_null NOT NULL hora_inicio,  
  CONSTRAINT disponibilidad_hora_fin_not_null NOT NULL hora_fin  
);
```

```
-- Permissions
```

```
ALTER TABLE prototipo.disponibilidad OWNER TO postgres;  
GRANT ALL ON TABLE prototipo.disponibilidad TO postgres;
```

-- prototipo.tareas definition

-- Drop table

-- DROP TABLE prototipo.tareas;

```
CREATE TABLE prototipo.tareas (  
  titulo varchar(25) NOT NULL,  
  estado varchar(15) NOT NULL,  
  prioridad varchar(20) NOT NULL,  
  fecha_limite timestamp NOT NULL,  
  descripcion varchar(50) NULL,  
  CONSTRAINT tareas_estado_not_null NOT NULL estado,  
  CONSTRAINT tareas_fecha_limite_not_null NOT NULL fecha_limite,  
  CONSTRAINT tareas_pkey PRIMARY KEY (titulo),  
  CONSTRAINT tareas_prioridad_not_null NOT NULL prioridad,  
  CONSTRAINT tareas_titulo_not_null NOT NULL titulo  
);
```

-- Permissions

```
ALTER TABLE prototipo.tareas OWNER TO postgres;  
GRANT ALL ON TABLE prototipo.tareas TO postgres;
```

-- prototipo.ubicaciones definition

-- Drop table

-- DROP TABLE prototipo.ubicaciones;

```
CREATE TABLE prototipo.ubicaciones (  
  id_ubicacion serial4 NOT NULL,  
  nombre varchar(50) NOT NULL,  
  direccion varchar(100) NOT NULL,  
  ciudad varchar(30) NOT NULL,  
  capacidad int4 NOT NULL,  
  CONSTRAINT ubicaciones_capacidad_not_null NOT NULL capacidad,  
  CONSTRAINT ubicaciones_ciudad_not_null NOT NULL ciudad,  
  CONSTRAINT ubicaciones_direccion_not_null NOT NULL direccion,  
  CONSTRAINT ubicaciones_id_ubicacion_not_null NOT NULL id_ubicacion,  
  CONSTRAINT ubicaciones_nombre_not_null NOT NULL nombre,  
  CONSTRAINT ubicaciones_pkey PRIMARY KEY (id_ubicacion)  
);
```

-- Permissions

```
ALTER TABLE prototipo.ubicaciones OWNER TO postgres;  
GRANT ALL ON TABLE prototipo.ubicaciones TO postgres;
```

-- prototipo.usuarios definition

-- Drop table

-- DROP TABLE prototipo.usuarios;

```
CREATE TABLE prototipo.usuarios (  
id_usuario serial4 NOT NULL,  
nombre varchar(50) NOT NULL,  
apellido varchar(50) NOT NULL,  
fecha_registro date DEFAULT CURRENT_DATE NOT NULL,  
activo bool DEFAULT true NULL,  
CONSTRAINT usuarios_apellido_not_null NOT NULL apellido,  
CONSTRAINT usuarios_fecha_registro_not_null NOT NULL fecha_registro,  
CONSTRAINT usuarios_id_usuario_not_null NOT NULL id_usuario,  
CONSTRAINT usuarios_nombre_not_null NOT NULL nombre,  
CONSTRAINT usuarios_pkey PRIMARY KEY (id_usuario)  
);
```

-- Permissions

```
ALTER TABLE prototipo.usuarios OWNER TO postgres;  
GRANT ALL ON TABLE prototipo.usuarios TO postgres;
```

-- prototipo.categorias definition

-- Drop table

-- DROP TABLE prototipo.categorias;

```
CREATE TABLE prototipo.categorias (  
id_categoria serial4 NOT NULL,  
nombre varchar(50) NOT NULL,  
id_categoria_padre int4 NULL,  
CONSTRAINT categorias_id_categoria_not_null NOT NULL id_categoria,  
CONSTRAINT categorias_nombre_not_null NOT NULL nombre,  
CONSTRAINT categorias_pkey PRIMARY KEY (id_categoria),  
CONSTRAINT categorias_id_categoria_padre_fkey FOREIGN KEY (id_categoria_padre) REFERENCES  
prototipo.categorias(id_categoria)  
);
```

-- Table Triggers

```
create trigger trg_evitar_ciclo before  
insert  
or  
update  
on  
prototipo.categorias for each row execute function prototipo.evitar_ciclo_categorias();
```

-- Permissions

```
ALTER TABLE prototipo.categorias OWNER TO postgres;  
GRANT ALL ON TABLE prototipo.categorias TO postgres;
```

```
-- prototipo.eventos definition
```

```
-- Drop table
```

```
-- DROP TABLE prototipo.eventos;
```

```
CREATE TABLE prototipo.eventos (  
  id_evento serial4 NOT NULL,  
  id_usuario_propietario int4 NOT NULL,  
  id_categoria int4 NOT NULL,  
  titulo varchar(100) NOT NULL,  
  descripcion text NULL,  
  fecha_inicio timestamp NOT NULL,  
  fecha_fin timestamp NOT NULL,  
  id_ubicacion int4 NULL,  
  CONSTRAINT check_fechas CHECK ((fecha_fin > fecha_inicio)),  
  CONSTRAINT eventos_fecha_fin_not_null NOT NULL fecha_fin,  
  CONSTRAINT eventos_fecha_inicio_not_null NOT NULL fecha_inicio,  
  CONSTRAINT eventos_id_categoria_not_null NOT NULL id_categoria,  
  CONSTRAINT eventos_id_evento_not_null NOT NULL id_evento,  
  CONSTRAINT eventos_id_usuario_propietario_not_null NOT NULL id_usuario_propietario,  
  CONSTRAINT eventos_pkey PRIMARY KEY (id_evento),  
  CONSTRAINT eventos_titulo_not_null NOT NULL titulo,  
  CONSTRAINT eventos_id_categoria_fkey FOREIGN KEY (id_categoria) REFERENCES  
  prototipo.categorias(id_categoria),  
  CONSTRAINT eventos_id_usuario_propietario_fkey FOREIGN KEY (id_usuario_propietario) REFERENCES  
  prototipo.usuarios(id_usuario),  
  CONSTRAINT id_ubicacion_fk FOREIGN KEY (id_ubicacion) REFERENCES prototipo.ubicaciones(id_ubicacion)  
);
```

```
-- Permissions
```

```
ALTER TABLE prototipo.eventos OWNER TO postgres;  
GRANT ALL ON TABLE prototipo.eventos TO postgres;
```

```
-- prototipo.log_accesos definition
```

```
-- Drop table
```

```
-- DROP TABLE prototipo.log_accesos;
```

```
CREATE TABLE prototipo.log_accesos (  
  id_log serial4 NOT NULL,  
  id_usuario int4 NULL,  
  fecha_acceso timestamp DEFAULT CURRENT_TIMESTAMP NULL,  
  CONSTRAINT log_accesos_id_log_not_null NOT NULL id_log,  
  CONSTRAINT log_accesos_pkey PRIMARY KEY (id_log),
```

```
CONSTRAINT log_accesos_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES
prototipo.usuarios(id_usuario)
);
```

```
-- Permissions
```

```
ALTER TABLE prototipo.log_accesos OWNER TO postgres;
GRANT ALL ON TABLE prototipo.log_accesos TO postgres;
```

```
-- prototipo.participaciones definition
```

```
-- Drop table
```

```
-- DROP TABLE prototipo.participaciones;
```

```
CREATE TABLE prototipo.participaciones (
id_evento int4 NOT NULL,
id_invitado int4 NOT NULL,
rol varchar(50) NULL,
estado_confirmacion varchar(20) DEFAULT 'pendiente'::character varying NULL,
CONSTRAINT participaciones_id_evento_not_null NOT NULL id_evento,
CONSTRAINT participaciones_id_invitado_not_null NOT NULL id_invitado,
CONSTRAINT participaciones_pkey PRIMARY KEY (id_evento, id_invitado),
CONSTRAINT participaciones_id_evento_fkey FOREIGN KEY (id_evento) REFERENCES
prototipo.eventos(id_evento) ON DELETE CASCADE,
CONSTRAINT participaciones_id_invitado_fkey FOREIGN KEY (id_invitado) REFERENCES
prototipo.usuarios(id_usuario)
);
```

```
-- Permissions
```

```
ALTER TABLE prototipo.participaciones OWNER TO postgres;
GRANT ALL ON TABLE prototipo.participaciones TO postgres;
```

```
-- prototipo.usuario_emails definition
```

```
-- Drop table
```

```
-- DROP TABLE prototipo.usuario_emails;
```

```
CREATE TABLE prototipo.usuario_emails (
id_usuario int4 NOT NULL,
email varchar(100) NOT NULL,
CONSTRAINT usuario_emails_email_not_null NOT NULL email,
CONSTRAINT usuario_emails_id_usuario_not_null NOT NULL id_usuario,
CONSTRAINT usuario_emails_pkey PRIMARY KEY (id_usuario, email),
CONSTRAINT usuario_emails_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES
prototipo.usuarios(id_usuario)
);
```

-- Permissions

```
ALTER TABLE prototipo.usuario_emails OWNER TO postgres;
GRANT ALL ON TABLE prototipo.usuario_emails TO postgres;
```

-- prototipo.usuario_telefonos definition

-- Drop table

-- DROP TABLE prototipo.usuario_telefonos;

```
CREATE TABLE prototipo.usuario_telefonos (
id_usuario int4 NOT NULL,
telefono varchar(20) NOT NULL,
CONSTRAINT usuario_telefonos_id_usuario_not_null NOT NULL id_usuario,
CONSTRAINT usuario_telefonos_pkey PRIMARY KEY (id_usuario, telefono),
CONSTRAINT usuario_telefonos_telefono_not_null NOT NULL telefono,
CONSTRAINT usuario_telefonos_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES
prototipo.usuarios(id_usuario)
);
```

-- Permissions

```
ALTER TABLE prototipo.usuario_telefonos OWNER TO postgres;
GRANT ALL ON TABLE prototipo.usuario_telefonos TO postgres;
```

-- prototipo.vista_antiguedad_usuarios source

```
CREATE OR REPLACE VIEW prototipo.vista_antiguedad_usuarios
AS SELECT id_usuario,
nombre,
fecha_registro,
age(CURRENT_DATE::timestamp with time zone, fecha_registro::timestamp with time zone) AS antiguedad
FROM prototipo.usuarios;
```

-- Permissions

```
ALTER TABLE prototipo.vista_antiguedad_usuarios OWNER TO postgres;
GRANT ALL ON TABLE prototipo.vista_antiguedad_usuarios TO postgres;
```

-- prototipo.vista_duracion_eventos_diarios source

```
CREATE OR REPLACE VIEW prototipo.vista_duracion_eventos_diarios
AS SELECT id_usuario_propietario,
fecha_inicio::date AS dia,
sum(EXTRACT(epoch FROM fecha_fin - fecha_inicio) / 60::numeric) AS duracion_total_minutos
FROM prototipo.eventos
GROUP BY id_usuario_propietario, (fecha_inicio::date);
```

-- Permissions

```
ALTER TABLE prototipo.vista_duracion_eventos_diarios OWNER TO postgres;  
GRANT ALL ON TABLE prototipo.vista_duracion_eventos_diarios TO postgres;
```

-- DROP FUNCTION prototipo.evitar_ciclo_categorias();

```
CREATE OR REPLACE FUNCTION prototipo.evitar_ciclo_categorias()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $function$  
BEGIN  
IF NEW.id_categoria_padre = NEW.id_categoria THEN  
RAISE EXCEPTION 'Una categoría no puede ser padre de sí misma.';  
END IF;  
-- Aquí se podría añadir una consulta recursiva para validar ancestros,  
-- pero para Postgres 14 es altamente eficiente usar el camino (path) o este chequeo simple.  
RETURN NEW;  
END;  
$function$  
;
```

-- Permissions

```
ALTER FUNCTION prototipo.evitar_ciclo_categorias() OWNER TO postgres;  
GRANT ALL ON FUNCTION prototipo.evitar_ciclo_categorias() TO postgres;
```

-- Permissions

```
GRANT ALL ON SCHEMA prototipo TO postgres;
```

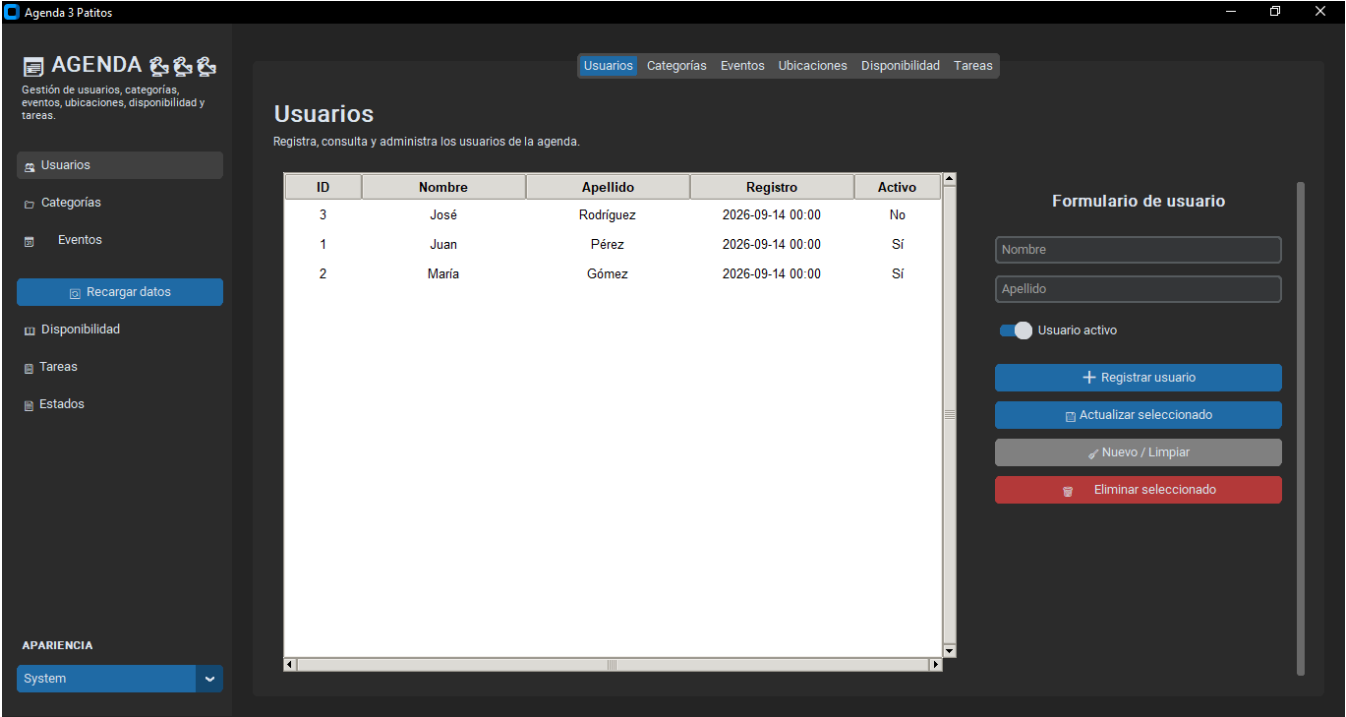
2. Estado de implementación (plantilla de lista de tareas):

Requerimiento	Fase	Tareas	Estado
RF-08	1. Modelo Conceptual	Definir la entidad ubicaciones y sus atributos obligatorios desde la interfaz: id_ubicacion (PK), nombre, direccion, ciudad y capacidad.	Listo
	2. Modelo Lógico	Estructurar la tabla relacional ubicaciones especificando tipos de datos y su clave primaria para la administración desde la GUI.	Listo
	3. Modelo Físico	Codificar el comando CREATE TABLE ubicaciones en PostgreSQL asegurando restricciones de nulidad y tipos óptimos.	Listo
	4. Interfaz Gráfica (GUI)	Desarrollar la pestaña y tabla de ubicaciones junto al formulario de entradas para registrar, consultar, actualizar y eliminar recintos.	Listo
RF-09	1. Modelo Conceptual	Establecer la relación de cardinalidad entre la entidad Eventos y la entidad ubicaciones mediante la clave foránea id_ubicacion.	Listo
	2. Modelo Lógico	Incorporar el atributo id_ubicacion como Clave Foránea (FK) en el esquema relacional de eventos para estructurar consultas complejas.	Listo
	3. Modelo Físico	Ejecutar la sentencia en PostgreSQL para añadir la relación referencial y soportar la detección de eventos simultáneos en un mismo recinto.	Listo
	4. Interfaz Gráfica (GUI)	Integrar consultas y selectores en la interfaz para consultar el histórico por ubicación y prevenir conflictos de espacio temporal.	Listo
RF-10	1. Modelo Físico	Diseñar y escribir las consultas para calcular el ranking de recintos más solicitados y con mayor volumen de eventos.	Listo
	2. Interfaz Gráfica (GUI)	Implementar un panel de reportes de solo lectura en la GUI para mostrar la tasa de uso de los espacios y la demanda operativa.	Listo
RF-11	1. Modelo Conceptual	Definir la entidad disponibilidades vinculada al perfil del usuario y el catálogo tipos_disponibilidad con sus atributos obligatorios de fecha y horas.	Listo
	2. Modelo Lógico	Estructurar las tablas relacionales para el control de disponibilidades y su clave foránea hacia usuarios.	Listo
	3. Modelo Físico	Codificar la creación de tablas y catálogos en PostgreSQL asegurando la integridad de las franjas temporales.	Listo
	4. Interfaz Gráfica (GUI)	Desarrollar el módulo en la interfaz para que los usuarios administren sus periodos libres u ocupados mediante operaciones CRUD completas.	Listo
RF-12	3. Modelo Físico	Estructurar las consultas analíticas en el motor de base de datos para la detección de cruces o traslapes temporales entre eventos y disponibilidades.	Listo
	4. Interfaz Gráfica (GUI)	Integrar las salidas analíticas en la GUI para determinar automáticamente usuarios libres y disponibles en un rango horario específico.	Listo

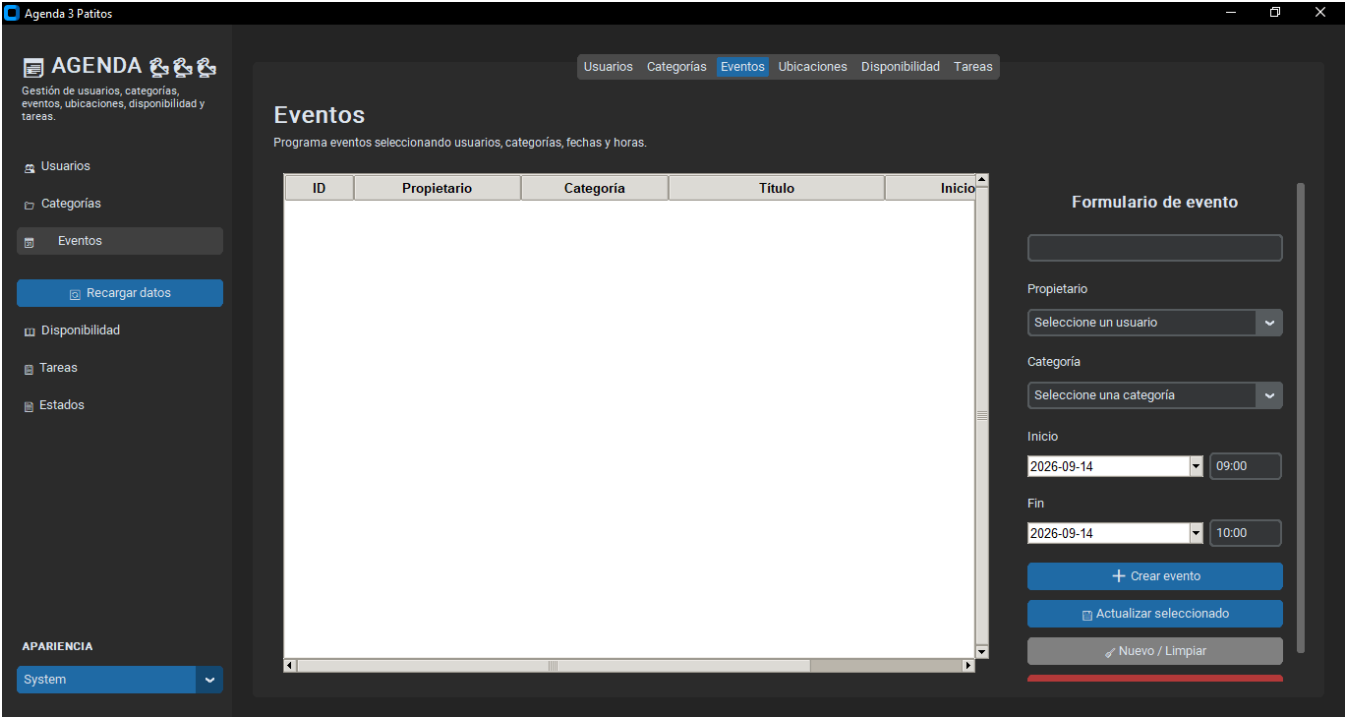
Requerimiento	Fase	Tareas	Estado
RF-15	1. Modelo Conceptual	Definir la entidad tareas vinculada a eventos con atributos de título, descripción, prioridad, fecha límite, responsable y estados.	Listo
	2. Modelo Lógico	Establecer las claves foráneas que relacionan cada tarea con su evento y usuario responsable.	Listo
	3. Modelo Físico	Ejecutar la creación de la tabla tareas en PostgreSQL con restricciones para los estados predefinidos.	Listo
	4. Interfaz Gráfica (GUI)	Habilitar formularios de control para que cada usuario realice un CRUD completo (crear, leer, actualizar estados y eliminar) desde la interfaz.	Listo
RF-16	3. Modelo Físico	Desarrollar las consultas orientadas a medir el rendimiento, tareas pendientes por usuario y detección de pendientes vencidos fuera de plazo.	Listo
	4. Interfaz Gráfica (GUI)	Configurar el despliegue de estas métricas operativas directamente en las pantallas de la aplicación.	Listo
RF-17	3. Modelo Físico	Estructurar la consulta del reporte analítico cuantitativo para filtrar tareas activas por estado y plazos vencidos.	Listo
	4. Interfaz Gráfica (GUI)	Integrar el reporte de seguimiento dentro de la interfaz del módulo de tareas para la detección temprana de sobrecargas de trabajo.	Listo

Parte 3.B: Evidencias visuales de la Interfaz Gráfica (GUI)

1. Gestión de Usuarios:



2. Gestión de Eventos:



3. Gestión de Categorías:

AGENDA

Gestión de usuarios, categorías, eventos, ubicaciones, disponibilidad y tareas.

Usuarios

Categorías

Eventos

Recargar datos

Disponibilidad

Tareas

Estados

APARIENCIA

System

UsuariosCategoríasEventosUbicacionesDisponibilidadTareas

Categorías

Organiza los eventos mediante categorías y subcategorías.

ID	Categoría	Categoría padre
1	Revisión	Sin categoría padre

Formulario de categoría

Nombre de la categoría

Categoría padre

Sin categoría padre

+ Crear categoría

Actualizar seleccionada

Nueva / Limpiar

Eliminar seleccionada

4. Gestión de Ubicaciones:

AGENDA

Gestión de usuarios, categorías, eventos, ubicaciones, disponibilidad y tareas.

Usuarios

Categorías

Eventos

Recargar datos

Disponibilidad

Tareas

Estados

APARIENCIA

System

UsuariosCategoríasEventosUbicacionesDisponibilidadTareas

Ubicaciones

Gestión de ubicaciones donde se realizarán los eventos.

ID	Nombre	Dirección	Ciudad	Capacidad
1	Centro de eventos	100m norte del estadio	Cartago	100

Formulario de ubicación

Nombre

Dirección

Ciudad

Capacidad

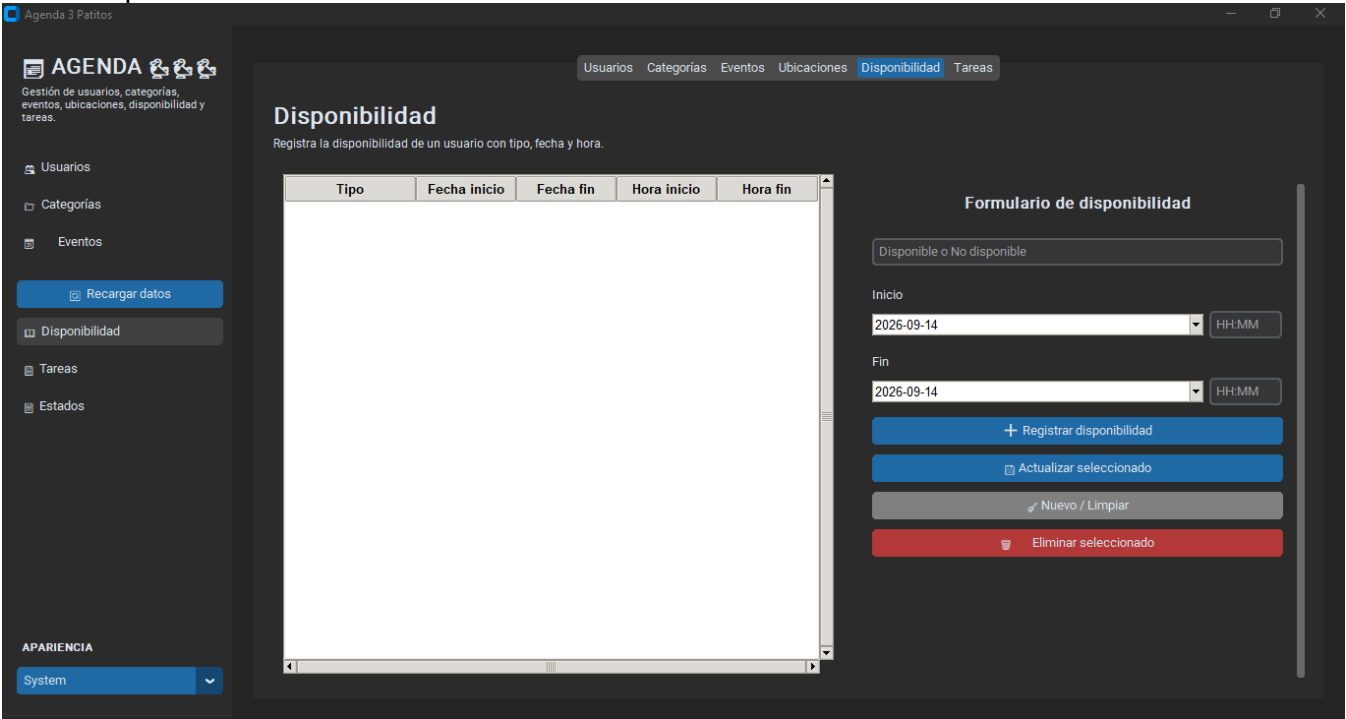
+ Registrar ubicación

Actualizar seleccionado

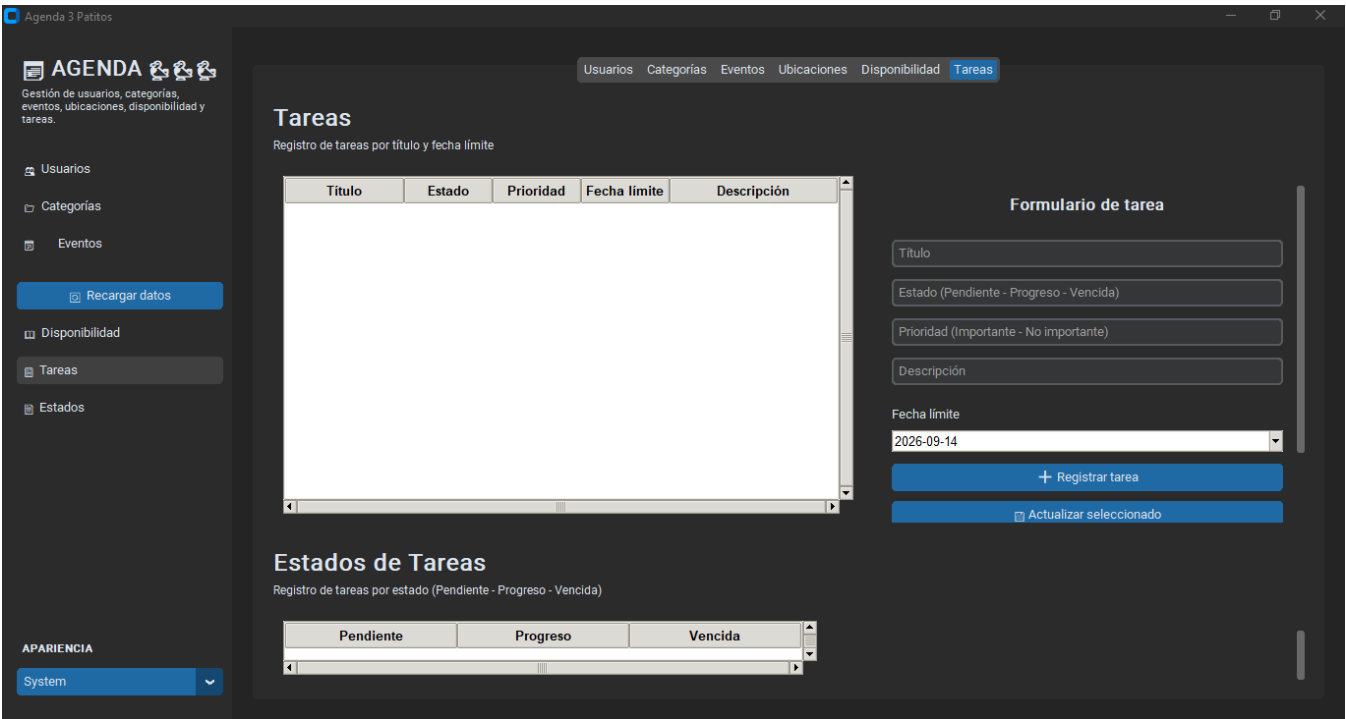
Nuevo / Limpiar

Eliminar seleccionado

5. Gestión de Disponibilidad:



6. Gestión de Tareas (y sección de estados):



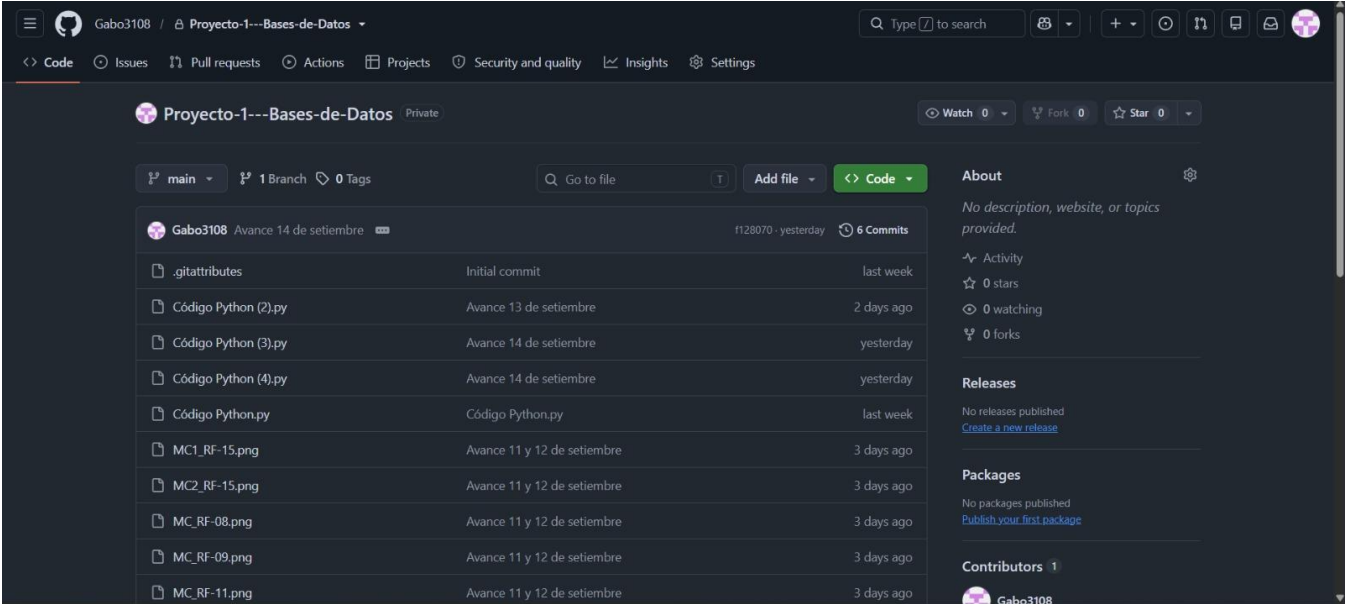
Parte 3.C: Repositorio en Git

Link de acceso:

El repositorio en Github se puede acceder mediante el siguiente enlace:

<https://github.com/Gabo3108/Proyecto-1---Bases-de-Datos.git>

Evidencias visuales:



Bibliografía

PostgreSQL Global Development Group. (s.f), *Queries with WITH*. PostgreSQL, <https://www.postgresql.org/docs/current/queries-with.html>

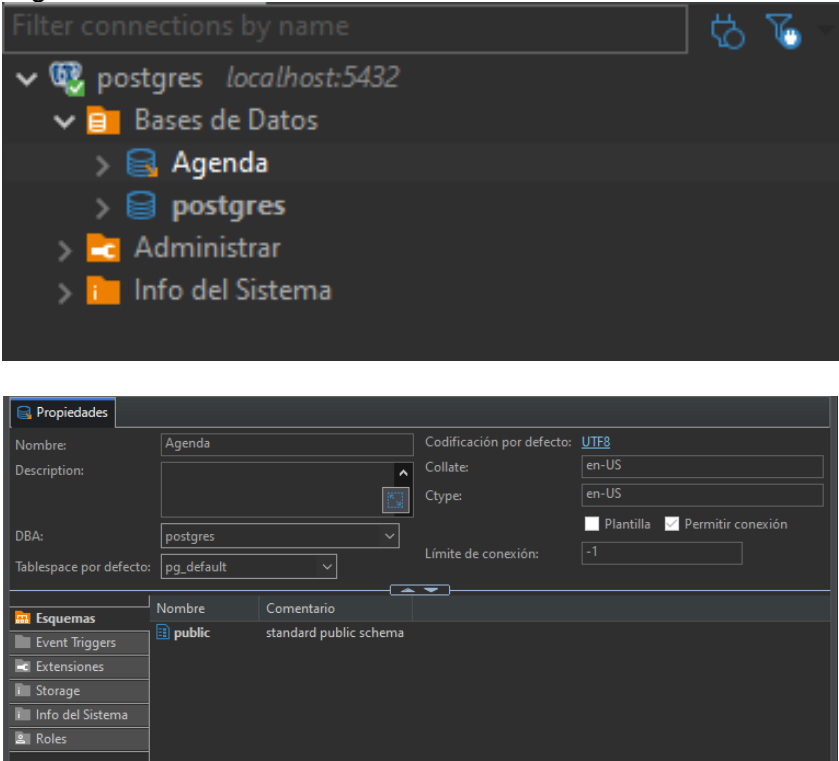
Python Software Foundation, (s.f), *tkinter message prompts*, Python documentation, <https://docs.python.org/es/3.10/library/tkinter.messagebox.html>

Python Software Foundation, (s.f), *sqlite3 – DB-API 2.0 interface for SQLite databases*, Python documentation, <https://docs.python.org/es/dev/library/sqlite3.html>

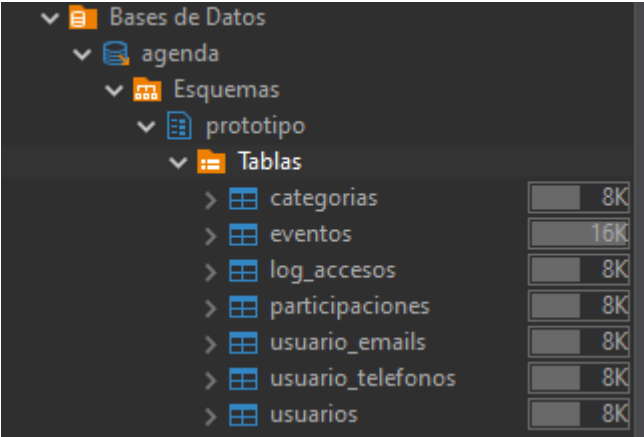
Anexos

1. Evidencias de Fase 1: Restauración de la Base de Datos.

Creación de base de datos “agenda”:



Tablas creadas:



Implementación de vistas:

```
-- Implementación de Cálculos Dinámicos (RF07, RE03, RN03) mediante vistas

-- Vista para Antigüedad
CREATE VIEW vista_antigüedad_usuarios AS
SELECT
    id_usuario,
    nombre,
    fecha_registro,
    age(CURRENT_DATE, fecha_registro) AS antigüedad
FROM usuarios;
```

Name	Value
Updated Rows	0
Execute time	0,012s
Start time	Fri Sep 11 16:04:52 CST 2026
Finish time	Fri Sep 11 16:04:52 CST 2026
Query	-- Vista para Antigüedad CREATE VIEW vista_antigüedad_usuarios AS SELECT id_usuario, nombre, fecha_registro, age(CURRENT_DATE, fecha_registro) AS antigüedad FROM usuarios

```
-- Vista para Duración de eventos diarios
CREATE VIEW vista_duracion_eventos_diarios AS
SELECT
    id_usuario_propietario,
    fecha_inicio::DATE AS dia,
    SUM(EXTRACT(EPOCH FROM (fecha_fin - fecha_inicio)))/60 AS duracion_total_minutos
FROM eventos
GROUP BY id_usuario_propietario, fecha_inicio::DATE;
```

Name	Value
Updated Rows	0
Execute time	0,009s
Start time	Fri Sep 11 16:09:55 CST 2026
Finish time	Fri Sep 11 16:09:55 CST 2026
Query	-- Vista para Duración de eventos diarios CREATE VIEW vista_duracion_eventos_diarios AS SELECT id_usuario_propietario, fecha_inicio::DATE AS dia, SUM(EXTRACT(EPOCH FROM (fecha_fin - fecha_inicio)))/60 AS duracion_total_minutos FROM eventos GROUP BY id_usuario_propietario, fecha_inicio::DATE

Integridad y prevención de ciclos:

agenda

Script-10

```
--Integridad y Prevención de Ciclos (RE05)
--Para evitar ciclos en la jerarquía de categorías, podemos usar una función
--que verifique el ancestro antes de insertar o actualizar:

CREATE OR REPLACE FUNCTION evitar_ciclo_categorias()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.id_categoria_padre = NEW.id_categoria THEN
        RAISE EXCEPTION 'Una categoría no puede ser padre de sí misma.';
    END IF;
    -- Aquí se podría añadir una consulta recursiva para validar ancestros,
    -- pero para Postgres 14 es altamente eficiente usar el camino (path) o este chequeo simple.
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,219s
Start time	Fri Sep 11 16:13:19 CST 2026
Finish time	Fri Sep 11 16:13:19 CST 2026
Query	CREATE OR REPLACE FUNCTION evitar_ciclo_categorias() RETURNS TRIGGER AS \$\$ BEGIN IF NEW.id_categoria_padre = NEW.id_categoria THEN RAISE EXCEPTION 'Una categoría no puede ser padre de sí misma.'; END IF; -- Aquí se podría añadir una consulta recursiva para validar ancestros, -- pero para Postgres 14 es altamente eficiente usar el camino (path) o este chequeo simple. RETURN NEW; END; \$\$ LANGUAGE plpgsql

agenda

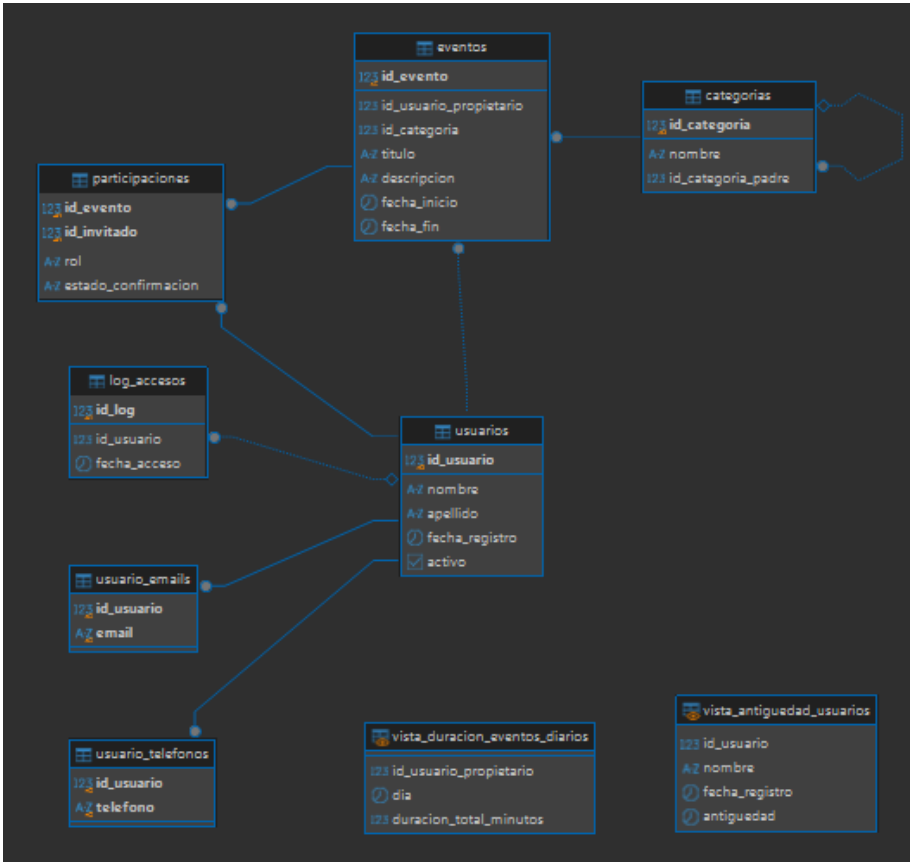
Script-10

```
CREATE TRIGGER trg_evitar_ciclo
BEFORE INSERT OR UPDATE ON categorias
FOR EACH ROW EXECUTE FUNCTION evitar_ciclo_categorias();
```

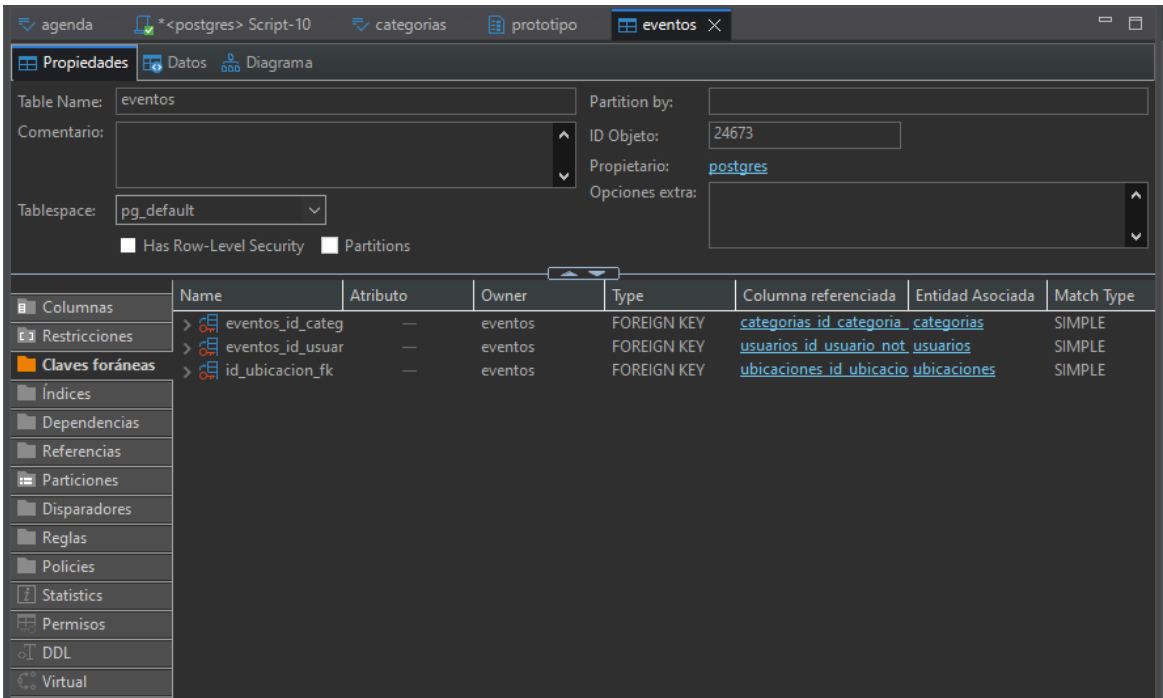
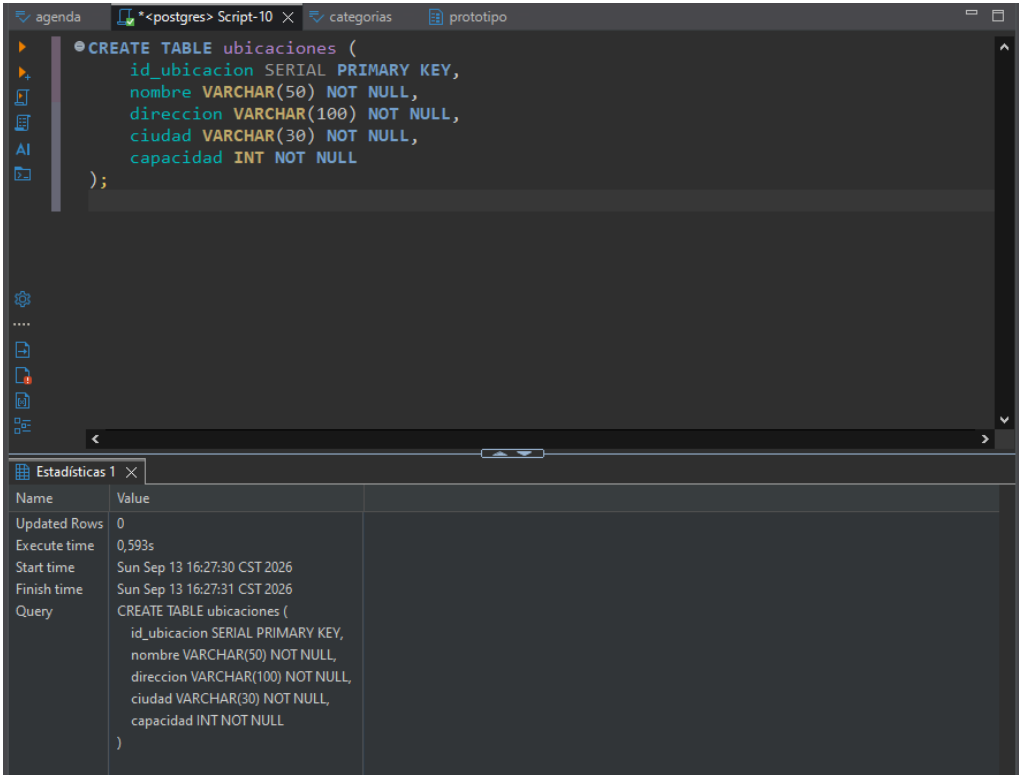
Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,019s
Start time	Fri Sep 11 17:21:59 CST 2026
Finish time	Fri Sep 11 17:21:59 CST 2026
Query	CREATE TRIGGER trg_evitar_ciclo BEFORE INSERT OR UPDATE ON categorias FOR EACH ROW EXECUTE FUNCTION evitar_ciclo_categorias()

Diagrama:



2. Evidencias de implementación de tablas en base de datos:



agenda

*<postgres> Script-10

categorias

prototipo

eventos

▶

⌵

📄

AI

📁

```
CREATE TABLE disponibilidad (  
    tipo_disponibilidad VARCHAR(15) PRIMARY KEY,  
    fecha TIMESTAMP NOT NULL,  
    hora_inicio TIMESTAMP NOT NULL,  
    hora_fin TIMESTAMP NOT NULL  
);
```

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,03s
Start time	Sun Sep 13 16:41:07 CST 2026
Finish time	Sun Sep 13 16:41:07 CST 2026
Query	CREATE TABLE disponibilidad (tipo_disponibilidad VARCHAR(15) PRIMARY KEY, fecha TIMESTAMP NOT NULL, hora_inicio TIMESTAMP NOT NULL, hora_fin TIMESTAMP NOT NULL)

agenda

*<postgres> Script-10

categorias

prototipo

eventos

▶

⌵

📄

AI

📁

```
CREATE TABLE tareas (  
    titulo VARCHAR(25) PRIMARY KEY,  
    estado VARCHAR(15) NOT NULL,  
    prioridad VARCHAR(20) NOT NULL,  
    fecha_limite TIMESTAMP NOT NULL,  
    descripcion VARCHAR(50)  
);
```

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,013s
Start time	Sun Sep 13 16:43:44 CST 2026
Finish time	Sun Sep 13 16:43:44 CST 2026
Query	CREATE TABLE tareas (titulo VARCHAR(25) PRIMARY KEY, estado VARCHAR(15) NOT NULL, prioridad VARCHAR(20) NOT NULL, fecha_limite TIMESTAMP NOT NULL, descripcion VARCHAR(50))

2. Declaración de uso de IA

Campo	Descripción
Ejercicio	Documento de código Python (configuración de la interfaz)
Herramientas utilizadas	ChatGPT-5 Luna
Etapa del trabajo	Comprensión del código y de librerías usadas en la interfaz
Descripción breve del uso	Se le solicitó a la IA explicar el funcionamiento, estructura y plantilla del código en cuanto a las funciones y operaciones relacionadas con las librerías que permiten mostrar la interfaz de las operaciones.
Declaración de autoría	"Declaro que comprendo y puedo explicar la totalidad del trabajo entregado, y que la IA fue utilizada únicamente como apoyo, no como sustituto de mi trabajo."